

Queues

Notes on the ADT, both implementations, and its classic applications, with Java and C/C++ examples

A note before you begin. A word before you read. These are my notes from years of teaching this unit, cleaned up so they are readable. While C/C++ was my first language, I am a bit rusty because I have taught Java for years now. For these notes, I have been using AI to help type these up, check for issues, and fill in explanations, which means a wrong or imprecise statement can slip in and survive longer than it should, and I do not fully trust the AI, so I am still editing. They are not meant to be a study guide, they are not gospel, and not everything here shows up in a lecture (and some lecture items are not here). If you find something that looks wrong, contradicts the book, or just does not behave the way I claim when you run it, tell me. You will be doing students and me a favor, and you will understand it better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. We are a two-language course, thus I will do my best to show everything in Java (CIS) and C/C++ (CS).

1. The queue ADT

A queue is an abstract data type, and the same wall idea from the linked list unit, and the same pattern used again in the stack unit, applies here too. We will not repeat that discussion, only apply it. A queue ADT specifies a small set of operations, enqueue, dequeue, front (or peek), isEmpty, and size, without saying anything about how those operations are carried out underneath. This unit covers two ways to carry them out, a circular array-based version and a linked version.

2. Terminology

- **FIFO.** First in, first out. The earliest item still in the queue is the first one removed.
- **Front.** The end of the queue where items are removed.
- **Rear.** The end of the queue where items are added. Some texts call this the back.
- **Enqueue.** Adds a new item at the rear.
- **Dequeue.** Removes and returns the item at the front.
- **Front (or peek).** Returns the item at the front without removing it.
- **Underflow.** An attempt to dequeue or peek an empty queue.
- **Full.** A fixed-capacity queue that cannot accept another enqueue until something is dequeued.

3. Circular array-based versus linked implementation

A plain array-based queue runs into a practical problem. As items are dequeued from the front and enqueued at the rear, both indices only ever move in one direction, so the slots before the front become unusable even though they are empty, and the queue eventually reports itself full while most of the array sits unused. A circular queue fixes this by wrapping an index back to 0 with the modulo operator once it runs past the last slot, so freed slots at the front get reused.

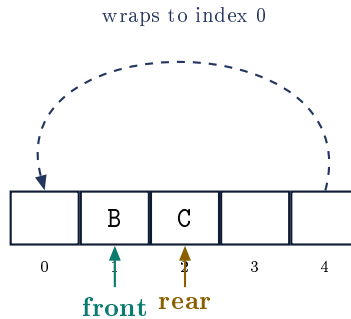


Figure 1: A circular array-based queue holding B and C. When rear reaches the last index, the next enqueue wraps it back to index 0 instead of running off the end of the array.

A linked queue avoids this problem a different way: it never resizes or wraps, since a new node is simply allocated for each enqueue and freed on each dequeue. It needs two references, front, exactly as in a linked stack's top, and rear, pointing at the last node, so that enqueue does not have to walk the whole chain to find where to attach the new node.

Table 1: Circular array-based versus linked implementations of a queue.

Aspect	Circular array-based	Linked
Memory layout	Contiguous block, front and rear indices wrap with modulo	Scattered nodes, a front reference and a rear (tail) reference
Capacity	Fixed, or resized once full	Grows and shrinks one node at a time, no fixed limit
Enqueue cost	$O(1)$, occasionally $O(n)$ on a resize	$O(1)$, because rear is tracked directly
Dequeue cost	$O(1)$	$O(1)$
Full versus empty	Both can leave front equal to rear, so a count or a sacrificed slot is needed to tell them apart	Naturally distinguished, front is null exactly when the queue is empty

A note on the code. The Java and C/C++ snippets below are not the exact code in your textbook, and I expect the two to look different in places. I simplify and reorganize for teaching purposes, and the book may name methods differently, split the class up differently, or handle a case, such as resizing or an empty queue, in a different order. These are snippets, not full listings, meant to show the key idea in each operation, not a complete class you can compile as-is. When the versions genuinely disagree, the textbook and the compiler settle it, not these notes.

4. Code snippets: circular array-based queue

Alongside the array, this version keeps a front index, a rear index, and a count of how many items are currently stored. The count is what actually distinguishes full from empty, since front and rear alone can land on the same index in both cases.

Java

```

private Object[] items;
private int front;
private int rear;
private int count;    // distinguishes full (count == items.length) from empty (count ==
                        0)

public void enqueue(T value) {
    if (count == items.length) {
        resize(items.length * 2);    // allocate a bigger array and re-lay out the items
    }
    items[rear] = value;
    rear = (rear + 1) % items.length;
    count++;
}

@SuppressWarnings("unchecked")
public T dequeue() {
    if (isEmpty()) {
        throw new NoSuchElementException("queue is empty");
    }
    T value = (T) items[front];
    items[front] = null;
    front = (front + 1) % items.length;
    count--;
    return value;
}

```

C/C++

```

T* items;
int capacity;
int front;
int rear;
int count;    // distinguishes full (count == capacity) from empty (count == 0)

void enqueue(const T& value) {
    if (count == capacity) {
        resize(capacity * 2);    // allocate a bigger array and re-lay out the items
    }
    items[rear] = value;
    rear = (rear + 1) % capacity;
    count++;
}

T dequeue() {
    if (isEmpty()) {
        throw std::underflow_error("queue is empty");
    }
    T value = items[front];
    front = (front + 1) % capacity;
    count--;
    return value;
}

```

```
}
```

5. Code snippets: linked queue

The linked queue keeps front, pointing at the oldest node, and rear, pointing at the newest, so enqueue only ever touches rear and dequeue only ever touches front.

Java

```
private Node<T> front;
private Node<T> rear;

public void enqueue(T value) {
    Node<T> newNode = new Node<>(value);
    if (isEmpty()) {
        front = newNode;
    } else {
        rear.setNext(newNode);
    }
    rear = newNode;
}

public T dequeue() {
    if (isEmpty()) {
        throw new NoSuchElementException("queue is empty");
    }
    T value = front.getData();
    front = front.getNext();
    if (front == null) {
        rear = null;    // the queue is now empty; do not leave rear dangling
    }
    return value;
}
```

C/C++

```
Node<T>* front;
Node<T>* rear;

void enqueue(const T& value) {
    Node<T>* newNode = new Node<T>(value);
    if (isEmpty()) {
        front = newNode;
    } else {
        rear->next = newNode;
    }
    rear = newNode;
}

T dequeue() {
```

```

    if (isEmpty()) {
        throw std::underflow_error("queue is empty");
    }
    Node<T>* oldFront = front;
    T value = oldFront->data;
    front = front->next;
    if (front == nullptr) {
        rear = nullptr; // the queue is now empty; do not leave rear dangling
    }
    delete oldFront; // free the node we just unlinked
    return value;
}

```

6. Applications, in pseudocode

Simulating a line with arrivals and departures

```

create an empty queue to represent the line
for each event in the arrival/departure log, in time order:
    if the event is an arrival:
        enqueue the new customer
    else if the event is a departure:
        dequeue the customer at the front and record how long they waited
at the end, the queue's remaining contents are whoever is still waiting

```

Preview: level-by-level traversal of a graph

Later in the course, when we cover graphs, the standard way to visit every node level by level, called breadth-first search, uses a queue to decide what to visit next. This is only a preview; we are not building a graph in this unit.

```

create an empty queue
mark the start node as visited, and enqueue it
while the queue is not empty:
    dequeue a node, call it current
    visit current
    for each neighbor of current that has not been visited:
        mark the neighbor as visited
        enqueue the neighbor

```

7. Traps people fall into

- **Dequeuing or peeking an empty queue.** Always check isEmpty first, or catch the exception the operation throws.
- **Telling full from empty by index alone.** In a circular array-based queue, front equal to rear happens both when the queue is empty and when it is completely full. Keep a separate count, or deliberately leave one slot always unused, to tell the two apart.

- **Off-by-one in the modulo arithmetic.** Advancing an index without the modulo (`front + 1` instead of `(front + 1) % capacity`) runs the index straight off the end of the array instead of wrapping it.
- **Forgetting to set rear on the first enqueue.** When a linked queue is empty, enqueue has to set both front and rear to the new node, not just link it after a rear that does not exist yet.
- **Leaving rear dangling after the last dequeue.** Once the last node is dequeued from a linked queue, front becomes null, but rear still points at the node that was just removed unless you explicitly null it out too.
- **Forgetting to delete in C/C++.** Dequeuing a node does not free its memory by itself. If you are not using smart pointers, you still need to call delete on the node you just unlinked, or the memory leaks.

8. Where this lives in the textbooks

Both books use the wall to name the ADT chapter, so it is easy to find.

- **Java, 3rd edition (Prichard and Carrano).** Chapter 8, Queues, follows the same pattern as the stack chapter before it: the ADT itself, applications, and array-based and reference-based implementations with a comparison.
- **C/C++, 8th edition (Carrano and Henry).** Chapter 10, Queues and Deques, covers the ADT queue (10.1), applications of the ADT queue (10.2), and the related ADT deque, a queue that allows insertion and removal at both ends (10.3 and 10.4). Chapter 11, Queue and Deque Implementations, covers implementations of the ADT queue (11.1), a linked implementation of the ADT deque (11.2), and a comparison of implementations (11.3).